

Colbeef

Documentación Técnica — Sistema LockerBeef

Documentación integral del código, la arquitectura, los lenguajes y la lógica de negocio del portal de control de lockers, dotaciones y personal (**LockerBeef** / gestor_lockers) de Colbeef.

Autor de referencia técnica: equipo de Tecnología Colbeef

Versión del documento: 1.0 · Producto: LockerBeef

Repositorio: <https://github.com/brayang466/gestion-lockers>

Tabla de contenido

- 1. Resumen ejecutivo
- 2. Lenguajes y stack tecnológico
- 3. Arquitectura general
- 4. Estructura del proyecto
- 5. Configuración y variables de entorno
- 6. Modelo de datos (base de datos)
- 7. Autenticación, sesión y seguridad
- 8. Roles, módulos y control de acceso
- 9. Módulos funcionales
- 10. Flujos operativos
- 11. Lógica de negocio destacada
- 12. Sistema de correo
- 13. Usabilidad (auditoría de navegación)
- 14. Capa de presentación (frontend)
- 15. Referencia de rutas (endpoints)
- 16. Despliegue y ejecución
- 17. Convenciones y notas de mantenimiento (nivel senior)

1. Resumen ejecutivo

El **Sistema LockerBeef Colbeef** es una aplicación web monolítica construida sobre **Python + Flask** con persistencia en **MySQL** (ORM Flask-SQLAlchemy). Centraliza el control de lockers, dotaciones y personal por área de trabajo:

- **Inventario de lockers y dotaciones:** bases, disponibles, ingresos y seca-botas.
- **Asignaciones y personal:** personal pendiente, registro de asignaciones, personal presupuestado.
- **Operaciones:** historial de retiros y formularios de ingreso.
- **Multi-área:** selector de área de sesión (DESPOSTE, BENEFICIO, CALIDAD, LOGISTICA, EXTERNOS/OTRAS AREAS, etc.).
- **Administración:** gestión de usuarios y roles (solo superadmin).
- **Usabilidad:** registro automático de navegación por usuario (hora, fecha, área, acción).
- **Mantenimiento de área:** bloqueo de DESPOSTE para usuarios distintos de superadmin.

Características transversales:

- Recuperación de contraseña por correo electrónico (SMTP).
- Cierre de sesión por inactividad (25 minutos).
- Favicon e identidad visual LockerBeef / Colbeef.
- Dashboard con estadísticas en tiempo real (polling HTTP).
- Dashboard React opcional (Vite) servido como estáticos.

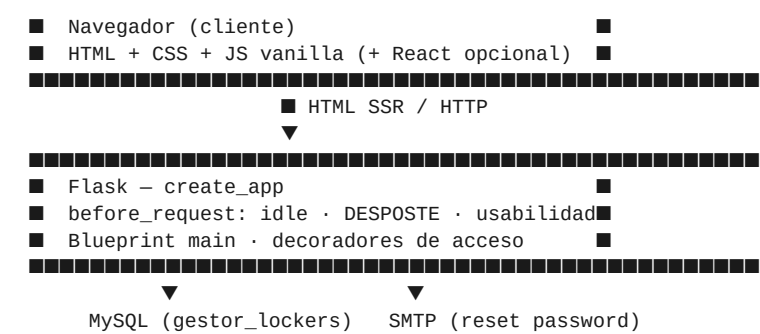
2. Lenguajes y stack tecnológico

Capa	Tecnología	Uso principal
Backend	Python 3.x	Lógica de negocio, rutas HTTP, scripts
Framework web	Flask ≥ 3.0	Enrutamiento, sesiones, Application Factory
ORM	Flask-SQLAlchemy ≥ 3.1	Persistencia relacional tipada
Plantillas	Jinja2	HTML del lado del servidor (SSR)
Base de datos	MySQL 8+ (utf8mb4)	Persistencia
Driver BD	PyMySQL	Conexión MySQL vía SQLAlchemy
Correo	smtpplib + email	Envío SMTP (TLS / SSL)
Config	python-dotenv	Variables desde .env
Passwords	Werkzeug	Hash y verificación
Tokens	itsdangerous	Restablecimiento de contraseña
Frontend principal	HTML5, CSS3, JS vanilla	UI responsiva SSR
Tipografías	Plus Jakarta Sans	Identidad visual
Dashboard opcional	React 18 + Vite 5 + Tailwind	Build en /static/dashboard/
Excel / import	openpyxl	Scripts de importación
Entrada desarrollo	run.py	app.run con host/puerto

Principio de diseño: monolito modular con un blueprint (**main**) y configuración de módulos (**MODULOS_CONFIG**). La UI operativa es Jinja2 + CSS + JS vanilla. El tiempo real del dashboard usa polling HTTP hacia **/dashboard/api/stats**.

3. Arquitectura general





Patrón: monolito modular. Las rutas orquestan validación, filtrado por área y renderizado. La configuración de cada módulo CRUD vive en **MODULOS_CONFIG**.

Ciclo de vida de una petición:

- `_idle_timeout_guard` actualiza actividad, aplica mantenimiento DESPOSTE y registra usabilidad (omite /static/).
- La ruta valida acceso (login, área actual, gate superadmin si aplica).
- La vista aplica filtros de dominio y renderiza Jinja2 o responde JSON.
- El cliente consume CSS/JS y puede refrescar stats vía API.

4. Estructura del proyecto

```
gestor_lockers/
■■■■ run.py                # Arranque desarrollo
■■■■ build_dashboard.bat   # Build React
■■■■ requirements.txt      #
■■■■ .env                  # Secretos (no versionar)
■■■■ app/
■   ■■■■ __init__.py       # create_app
■   ■■■■ config.py         #
■   ■■■■ models/           # ORM
■   ■■■■ routes/main.py    # Núcleo de negocio
■   ■■■■ utils/email.py    #
■   ■■■■ templates/        # Jinja2
■   ■■■■ static/           # css, js, favicon, dashboard/
■■■■ frontend/            # Fuente React + Vite
■■■■ database/             # SQL / dumps
■■■■ scripts/              # admin, import, export
■■■■ docs/                 # Documentación
■■■■ datos_importar/       # CSV
```

5. Configuración y variables de entorno

app/config.py centraliza la configuración mediante la clase **Config**, poblada desde **.env** (python-dotenv).

Variable	Descripción	Default / notas
SECRET_KEY	Firma de sesión	Obligatorio en producción
PORT	Puerto de run.py	5000
MYSQL_HOST / PORT	Conexión MySQL	127.0.0.1 / 3306
MYSQL_USER / PASSWORD	Credenciales BD	root / vacío
MYSQL_DATABASE	Nombre de base	gestor_lockers
MAIL_SERVER / HOST	Servidor SMTP	smtp.gmail.com
MAIL_PORT	Puerto SMTP	587 TLS / 465 SSL
MAIL_USERNAME / PASSWORD	Credenciales SMTP	—

Variable	Descripción	Default / notas
MAIL_DEFAULT_SENDER / FROM	Remitente	Fallback a username
MAIL_FROM_NAME	Nombre visible	LockerBeef
PASSWORD_RESET_EXPIRE_MINUTES	Vigencia del enlace	15
APP_URL	URL pública (correos)	http://192.168.x.x:5000

Seguridad: .env nunca debe versionarse con secretos reales. SECRET_KEY débil solo es aceptable en desarrollo local.

6. Modelo de datos (base de datos)

Motor MySQL, utf8mb4. Tablas con **db.create_all()** al arrancar; evolución con scripts en **database/** y modelos ORM.

6.1 Usuarios y áreas

Tabla	Descripción
usuarios	Cuentas; UK email; roles; área; activo
area_trabajo	Catálogo de áreas (nombre UNIQUE, UPPER)
usabilidad_log	Auditoría de navegación; FK opcional a usuarios

6.2 Lockers y dotaciones

Tabla	Descripción
base_lockers	Inventario maestro de lockers
locker_disponibles	Lockers disponibles
base_dotaciones	Inventario maestro de dotaciones
dotaciones_disponibles	Soporte / legado
seca_botas_disponibles	Códigos de seca-botas
ingreso_lockers / ingreso_dotacion	Altas de ingreso

6.3 Personal y operaciones

Tabla	Descripción
registro_personal	Registro de personal
registro_asignaciones	Asignaciones / pendiente; flag es_planta_desposte
historial_retiros	Retiros; flag es_planta_desposte
personal_presupuestado	Cupos por área

6.4 Diagrama entidad-relación (resumen)

```

usuarios ■■■< usabilidad_log
area_trabajo (catálogo de sesión)
base_lockers / locker_disponibles / seca_botas_disponibles
base_dotaciones / registro_asignaciones / historial_retiros
personal_presupuestado / ingreso_* / registro_personal

```

Aislamiento multi-área por columnas (area, area_uso, subarea, es_planta_desposte).

7. Autenticación, sesión y seguridad

7.1 Acceso a datos

Acceso a MySQL mediante Flask-SQLAlchemy (**db.session**, modelos en **app/models/**).

7.2 Inicio de sesión

/login valida email + activo + **check_password_hash**. Al autenticar setea **user_id**, **user_nombre**, **user_email**, **user_rol**, **user_area**; limpia **current_area** y redirige a **/areas**. Existe también **/acceso-integrado**.

7.3 Gestión de sesión

- Idle timeout: **25 minutos**.
- Reloj en la barra de navegación.
- Logout: GET **/logout** limpia sesión.

7.4 Reset de contraseña

Tokens firmados con **itsdangerous**, vigencia configurable. Requiere SMTP. Guía: docs/RECUPERACION_CONTRASENA.md.

7.5 Controles adicionales

- Passwords hasheados (Werkzeug).
- Rutas admin con **@_superadmin_required**.
- DESPOSTE bloqueado en mantenimiento salvo superadmin.

8. Roles, módulos y control de acceso

8.1 Roles

Rol	Código	Capacidades
Super Administrador	superadmin	Todas las áreas; edición; usuarios; usabilidad; DESPOSTE
Administrador	admin	Todas las áreas; edición; sin usuarios/usabilidad
Coordinador	coordinador	Su área; edición
Usuario	usuario	Su área; solo consulta

8.2 Control de acceso

Mecanismo	Efecto
@login_required	Exige sesión
@_require_current_area	Exige área elegida
@_superadmin_required	Solo superadmin
_user_can_edit()	Permite mutaciones en módulos
_allowed_areas_for_user()	Áreas del selector

8.3 Matriz resumida

Capacidad	Usuario	Coord.	Admin	Superadmin
Selector de áreas	✓	✓	✓ todas	✓ todas
Dashboard / consulta	✓	✓	✓	✓
Editar módulos	✗	✓	✓	✓
Gestión de usuarios	✗	✗	✗	✓
Usabilidad	✗	✗	✗	✓

Capacidad	Usuario	Coord.	Admin	Superadmin
DESPOSTE (mantenimiento)	X	X	X	✓

9. Módulos funcionales

- **Selector de áreas** (/areas) con modal de mantenimiento para DESPOSTE.
- **Dashboard** (/dashboard) con KPIs y sidebar.
- **Inventario**: Base Lockers, Locker Disponibles, Seca Botas, Base Dotaciones, Dotaciones Disponibles.
- **Ingresos** (nav): Ingreso Lockers, Ingreso Dotación, Registro personal.
- **Personal/operaciones**: Personal Pendiente, Presupuestado, Asignaciones, Historial Retiros.
- **Superadmin**: Gestión de usuarios y Usabilidad por usuario.
- **Suite Principal**: enlace externo para admin/superadmin.

10. Flujos operativos

10.1 Acceso estándar

Login → /areas → /entrar-area/<NOMBRE> → session[current_area]
→ /dashboard → /dashboard/<modulo> (CRUD según rol)

10.2 DESPOSTE en mantenimiento

Usuarios distintos de superadmin reciben el anuncio: «*Esta área se encuentra en mantenimiento. Estará disponible luego.*» También se bloquea en servidor (/entrar-area/DESPOSTE) y se expulsan sesiones previas.

10.3 Usabilidad

Cada navegación autenticada (con omisiones y debounce) inserta un registro en **usabilidad_log**. Superadmin consulta /dashboard/usabilidad.

11. Lógica de negocio destacada

- **Scope por área**: helpers _lockers_por_sesion_filter, _registro_area_scope_filter, _base_dotaciones_scope_filter.
- **Flag es_planta_desposte**: separa planta Desposte de registros generales.
- **CRUD por configuración**: MODULOS_CONFIG + ruta /dashboard/<modulo_id>.
- **Hook unificado**: idle + usabilidad + mantenimiento DESPOSTE.
- **Verificación de códigos**: /dashboard/api/verificar-codigos.

12. Sistema de correo

Evento	Cuándo
Reset de contraseña	Usuario solicita recuperación
Notificación de cambio	Tras restablecer (si aplica)

Implementado en **app/utils/email.py**. Requiere MAIL_USERNAME, MAIL_PASSWORD y preferiblemente APP_URL.

13. Usabilidad (auditoría de navegación)

13.1 Captura automática

- Hook en before_request.
- Omite estáticos, login y APIs de polling.

- Debounce ~4 s por mismo path GET.
- Guarda usuario, rol, área, path, método, acción y timestamp.

13.2 Panel (superadmin)

/dashboard/usabilidad: filtros por usuario, fecha y texto; agrupación por día (máx. 500 registros); acciones legibles.

14. Capa de presentación (frontend)

- SSR con Jinja2; páginas extienden base.html.
- CSS: static/css/app.css y login.css.
- JS: app.js, login.js, dashboard-reminders.js.
- React opcional vía build_dashboard.bat.
- Favicon: static/favicon.svg.

Archivo	Función
app.js	Tema, navegación, utilidades
login.js	Validación / vistas login
dashboard-reminders.js	Recordatorios dashboard

15. Referencia de rutas (endpoints)

Autenticación

Ruta	Métodos	Función
/login	GET, POST	Inicio de sesión
/acceso-integrado	GET, POST	Login integrado
/logout	GET	Cerrar sesión
/registro	GET, POST	Alta de usuario
/restablecer-contrasena	GET, POST	Solicitar reset
/restablecer-contrasena/confirmar	GET, POST	Nueva contraseña

Áreas, dashboard y administración

Ruta	Métodos	Función
/	GET	Redirect según sesión
/areas	GET	Selector de área
/entrar-area/<path>	GET	Fija área
/dashboard	GET	Panel de control
/dashboard/api/stats	GET	KPIs JSON
/dashboard/api/verificar-codigos	GET	Disponibilidad códigos
/dashboard/<modulo_id>	GET, POST	CRUD genérico
/dashboard/registro/<modulo_id>	GET, POST	Formularios ingreso
/dashboard/usuarios	GET, POST	Usuarios (superadmin)
/dashboard/usabilidad	GET	Usabilidad (superadmin)

16. Despliegue y ejecución

16.1 Requisitos

- Python 3.x y MySQL 8+.
- `pip install -r requirements.txt` (venv recomendado).
- Node.js opcional si se reconstruye el dashboard React.

16.2 Base de datos

- Ejecutar `database/00_crear_base_vacia.sql`.
- Configurar `.env` (`MYSQL_*`).
- Arrancar app una vez (`db.create_all`).
- `python scripts/crear_admin.py`
- Opcional: `asignar_superadmin.py` e `importar_todo.py`.

16.3 Arranque

```
# Desarrollo
python run.py
# http://127.0.0.1:5000 o http://192.168.x.x:5000

# Build opcional React
build_dashboard.bat
```

17. Convenciones y notas de mantenimiento (nivel senior)

- **Filtros de área como fuente de verdad.** No listar inventarios sin helpers de scope.
- **Configurar módulos, no duplicar CRUD.** Nuevas tablas → modelo + `MODULOS_CONFIG`.
- **Roles normalizados.** Comparar siempre `.strip().lower()`.
- **Hook barato.** Respetar debounce de usabilidad y omisión de estáticos/APIs.
- **Esquema evolutivo.** `create_all()` no altera columnas; usar SQL + modelo.
- **Jinja y dicts.** Evitar claves `items/keys/values` en estructuras de plantilla.
- **Secretos fuera del repo.** Documentar nombres de variables, no valores.
- **Mantenimiento DESPOSTE.** Flag `DESPOSTE_EN_MANTENIMIENTO` en `main.py`.
- **Documentación dual.** Alinear `DOCUMENTACION_CODIGO.md` con este PDF al cambiar contratos.
- **Front React opcional.** Si se usa en producción, sincronizar con `MODULOS_CONFIG`.

Fin del documento — Documentación Técnica Sistema LockerBeef Colbeef · Versión 1.0